

# Tools Engineering Design Manual

## Governed Shared Calculations and Interchange Pathways

### Release Status

This is the canonical editable QMD source for the Tools engineering design manual. It is a governance scaffold, not a released engineering manual. The file-level module inventory is complete for its declared tracked-source denominator, but all detected calculation candidates remain provisional and the stable calculation-pathway registry remains empty. Release remains blocked by TOOLS-D3 through TOOLS-D9. Generated HTML, LaTeX, PDF, and DOCX artifacts are reproducible engineering records under TOOLS-D2, but none is approved for publication or engineering use.

### Documentation Authority and Release Boundary

#### Purpose

The manual will explain shared calculations and interchange contracts from theory and assumptions through typed inputs, units, coordinate and time conventions, algorithms, implementation symbols, tests, uncertainty, limitations, consumer surfaces, and artifacts. Tools source and evidence remain authoritative; the manual cannot approve a calculation that its owning source, evidence, license, or human review has not approved.

#### Canonical Source

Only QMD under `manuals/tools` is editable manual source. Existing user guides, ADRs, package documentation, generated API references, and publications are separate governed products. They can supply reviewed references, but none is a second mutable source for this manual.

Generated LaTeX, PDF, DOCX, and HTML are non-editable artifacts. A correction starts in canonical QMD, then regenerates every affected representation through the qualified toolchain. TOOLS-D2 owns that toolchain; TOOLS-D7 and TOOLS-D8 own artifact review and publication evidence.

#### Evidence and License Classes

Every calculation and claim must be classified as observed, derived, model-scenario, external, provisional, unavailable, or approved within an explicit scope. Every borrowed source and generated artifact must preserve its license and attribution. Content from `Tools_Private` or another non-public source may not enter this public manual without explicit public authority.

#### Approval and Failure Semantics

The default is deny-until-approved. Missing inventory coverage, source authority, units, coordinate or time convention, implementation symbol, test, validity range, uncertainty, license, artifact digest, page review, accessibility review, or human approval keeps the affected item blocked. Unsupported values are never replaced with synthetic or plausible values without an explicit model-scenario contract.

## Exemptions

Freshness failures are release-blocking. The TOOLS-D1 module baseline is recomputed from tracked implementation and governed configuration files by `python -m scripts.build_tools_module_inventory --check`; later TOOLS-D6 work must extend freshness from the file baseline to stable calculation pathways. Future exemptions must be structured, narrowly scoped, justified, owned, reviewed, and expiring. An exemption cannot authorize an equation, scientific claim, artifact, operating use, license transfer, or public projection.

## Current Boundary

TOOLS-D1 establishes a complete file-level denominator for its declared tracked scope. TOOLS-D2 establishes deterministic byte reproduction and governed semantic preservation across HTML, LaTeX, PDF, and DOCX. Its generated artifacts remain non-editable and unapproved. The conservative inventory does not establish stable calculation identity, equation correctness, validity ranges, uncertainty, page/accessibility approval, or publication authority. Those pathways remain empty and blocked pending TOOLS-D3 through TOOLS-D9.

## Module and Calculation-Candidate Inventory

### Purpose and Denominator

`manifests/module-inventory.json` is the deterministic TOOLS-D1 index for every tracked implementation source file and governed configuration module in the repository. It scans recognized Python, Python stub, JavaScript, TypeScript, Rust, MATLAB, C/C++, Arduino, shell, PowerShell, HTML, CSS, Prisma, JSON, YAML, TOML, INI, XML, URDF, and web-manifest files. Tests are evidence inputs rather than implementation rows. Generated, vendored, archived, experimental, example, documentation, resource, and manual trees are excluded explicitly in the manifest, so a consumer can reproduce the exact denominator. The index binds bounded entry shards by normalized SHA-256, contiguous index, entry count, and first/last path. Both the index and every shard remain below the repository's one-megabyte file budget; consumers reassemble one logical view only after verifying every shard and the exact declared shard set.

At this baseline, 3,434 modules are classified: 808 conservative calculation candidates and 2,626 non-calculation modules. One legacy JavaScript file has a non-UTF-8 encoding and remains explicitly blocked rather than being decoded by an implicit platform default. These counts describe inventory coverage only; they are not test coverage, calculation approval, or publication readiness.

### Deterministic Identity and Integrity

Each file receives a provisional path-derived identifier and a SHA-256 digest of normalized text bytes. The normalization maps CRLF and bare CR to LF before hashing. For module path ( $p_i$ ) and normalized content digest ( $h_i$ ), the tree authority is

$$H_{\text{tree}} = \text{SHA256}(\|_{i=1}^n p_i : h_i),$$

where rows are ordered by normalized repository-relative path. This contract makes the result independent of Windows checkout line endings while retaining content changes. The path-derived identifier is not a stable calculation ID; TOOLS-D3 owns stable equation-to-code identities that survive file movement.

### Classification Contract

A module is a calculation candidate only when its normalized path contains a governed calculation marker, such as dynamics, kinematics, pressure drop, signal processing, geometry, solver, energy, force, or trajectory, or when the source imports a recognized scientific mathematics library. All other modules are non-calculation.

This intentionally conservative static screen can have false positives and false negatives, so every calculation candidate has `authority_status=provisional` and `review_status=review-required`.

The inventory also records language, purpose where statically available, maintainer family, public surfaces and routes, mapped tests and validation, ADRs, equation references, literal units, citations, artifact digests, risk tags, normalized byte count, and content digest. Missing tests, sources, units, chapters, equation pathways, validation, routes, or artifact digests remain `unavailable`, `not-applicable`, or explicitly pending their owning subepic; they are never synthesized. Non-calculation status means only that the conservative detector found no calculation signal. It does not waive public-API, security, licensing, accessibility, or downstream compatibility review.

## Consumer and Failure Contract

The repository-owned schema is `schemas/module-inventory.schema.json`, version `tools-module-inventory/1.0.0`. It extends the program-owned calculation registry without copying or competing with that shared schema. Both JSON Schema validation and the Python consumer reject unknown fields, unsupported versions, unsafe paths, invalid hashes, duplicate identifiers, unsorted duplicate traceability arrays, denominator mismatches, and unsupported state values.

Any tracked in-scope module addition, removal, rename, or normalized content change makes `--check` fail until the manifest is regenerated and reviewed. A green inventory check proves only deterministic classification and freshness of this file-level baseline. Public projection remains blocked until the remaining stable-pathway, page, accessibility, licensing, publication, and human-approval gates are complete.

## Reproducible Rendering Contract

### Outcome and Authority

TOOLS-D2 renders the canonical QMD into deterministic HTML, LaTeX, PDF, and DOCX representations. Generated outputs are engineering records, not approval. Only QMD may be corrected directly. Every output remains blocked from public projection until semantic, page, accessibility, provenance, licensing, and human-review gates owned by later subepics are complete.

The renderer uses the pinned command versions and inputs recorded in `toolchain-lock.json`. It verifies the exact first version line before render, expands only bounded QMD includes, uses the governed bibliography and reference DOCX, disables TeX shell escape, normalizes volatile package metadata, and binds every representation into `manifests/artifacts.json`. The command model follows the documented Pandoc conversion contract (Pandoc contributors 2026) while QMD remains compatible with the pinned Quarto authoring surface (Posit Software, PBC 2026).

### Semantic, Unit, and Warning Preservation

The semantic contract is deliberately format-neutral. Every representation must retain the governed title, release boundary, representative unit string, warning, figure label, and bibliography identity before receiving a shared semantic digest. Example unit string: 10 m/s.

WARNING: Provisional calculations remain blocked.

This gate proves preservation of required meaning across the four representations. It does not prove calculation correctness, equation coverage, accessibility, page aesthetics, or release approval. Missing text, a stale digest, an unpinned tool, an unsafe include, or an absent artifact fails closed.

## Reproduction and Consumer Contract

Run `python -m scripts.render_tools_design_manual --verify-toolchain` before a qualified build. Run `python -m scripts.render_tools_design_manual --render` to replace renderer-owned artifacts,

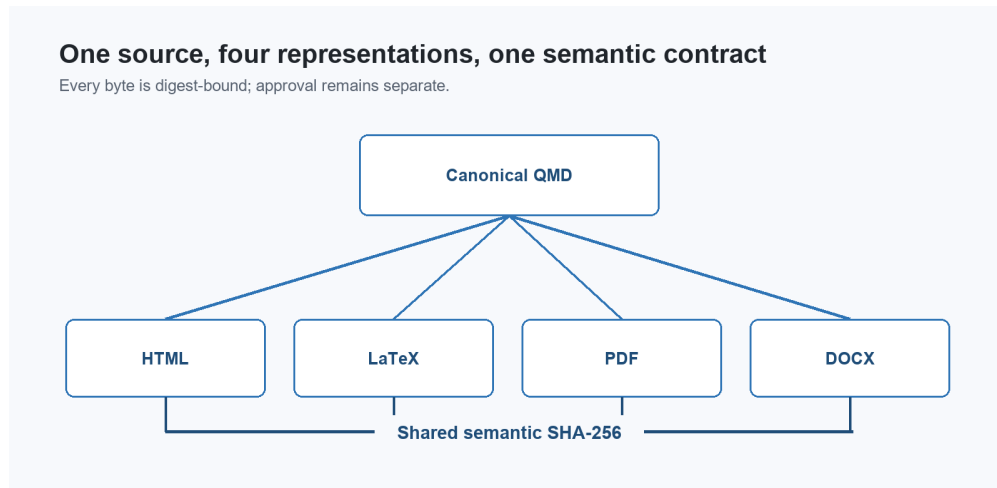


Figure 1: Deterministic rendering pathway.

then `python -m scripts.render_tools_design_manual --check` to validate source, toolchain, byte, and semantic bindings. The strict JSON Schemas allow consumers to validate the lock and manifest without importing producer code.

Artifact rows expose media type, byte count, byte digest, and shared semantic digest. The manifest also preserves the owning epic, review owner, explicit blockers, and a null immutable source commit while this local stack is not a merged authority. A consumer must reject unknown fields, versions, formats, paths, hashes, status promotion, or semantic divergence.

## References

Pandoc contributors. 2026. *Pandoc User's Guide*. Pandoc. <https://pandoc.org/MANUAL.html>.

Posit Software, PBC. 2026. *Quarto Guide*. Posit Software, PBC. <https://quarto.org/docs/guide/>.